

Documento Técnico - GS Generative AI - 1S 2026

Assistente RAG para documentos e dados da nova economia espacial.

Problema escolhido

Organizações, estudantes e equipes técnicas que trabalham com a nova economia espacial precisam consultar documentos extensos sobre clima, satélites, agricultura inteligente, monitoramento ambiental, desastres naturais e exploração espacial. A leitura manual desses materiais consome tempo e pode dificultar a tomada de decisão.

A solução proposta é um assistente inteligente com RAG que recupera trechos relevantes dos documentos antes de gerar uma resposta. Isso reduz respostas sem base documental e facilita o acesso ao conhecimento técnico.

Arquitetura da solução

A aplicação usa Next.js com rotas API para ingestão e pergunta. A interface envia arquivos para extração de texto, recebe chunks vetorizados e mantém uma vector store em memória no navegador. Quando o usuário faz uma pergunta, os chunks são enviados para a API de pergunta, que faz a busca semântica e chama o modelo generativo da Groq com o contexto recuperado.

Fluxo resumido:

- Upload ou uso de documentos de exemplo.
- Extração de texto de PDF, TXT ou DOCX.
- Normalização e divisão em chunks.
- Geração de embeddings com OpenAI quando configurado, ou fallback local por hashing vetorial.
- Busca vetorial por similaridade cosseno.
- Montagem do prompt com os trechos mais relevantes.
- Geração de resposta com Groq Llama 70B.
- Exibição da resposta com fontes citadas.

Ferramentas utilizadas

- Next.js, React e TypeScript para aplicação web.
- Tailwind CSS para interface.
- **pdf-parse** para extrair texto de PDFs.
- **mammoth** para extrair texto de DOCX.
- OpenAI Embeddings API para vetorização semântica dos uploads.
- **groq-sdk** para chamada ao modelo generativo.
- **lucide-react** para ícones da interface.

Modelo utilizado

O modelo generativo planejado é **llama-3.3-70b-versatile** via Groq API. A variável **GROQ_MODEL** permite trocar o modelo sem alterar o código caso a disponibilidade mude.

Vector store utilizada

Para a demonstração acadêmica, a vector store é local e em memória. Os documentos carregados são mantidos no estado da aplicação, junto com seus chunks e embeddings. Quando **OPENAI_API_KEY** está configurada, uploads usam **text-embedding-3-small**; caso contrário, a aplicação usa embeddings locais por hashing. Essa abordagem facilita a demonstração e evita depender de banco externo.

Em uma versão de produção, a vector store poderia ser substituída por Pinecone, Weaviate, Qdrant, Chroma, pgvector ou outro banco vetorial persistente.

Fluxo RAG implementado

O fluxo RAG implementado segue o padrão Retrieval-Augmented Generation:

- Retrieval: a pergunta é comparada com os embeddings dos chunks usando similaridade cosseno.
- Augmentation: os trechos mais relevantes são anexados ao prompt.
- Generation: o modelo da Groq gera uma resposta em português usando o contexto recuperado.

A resposta orienta o modelo a citar as fontes no formato **[Fonte 1]**, **[Fonte 2]** e a indicar limitações quando o contexto não for suficiente.

Limitações

- O fallback local por hashing serve para demonstração, mas não possui a mesma qualidade semântica de embeddings OpenAI.
- A vector store em memória é perdida ao recarregar a página.
- PDFs escaneados como imagem podem não ter texto extraível sem OCR.
- A qualidade da resposta depende da chave Groq, do modelo selecionado e dos documentos enviados.
- A aplicação não implementa autenticação nem persistência multiusuário.

Melhorias futuras

- Usar embeddings dedicados de um provedor externo ou modelo local.
- Persistir documentos e vetores em banco vetorial.
- Adicionar OCR para PDFs digitalizados.
- Permitir filtros por fonte, data, tema e tipo de documento.
- Exportar respostas e histórico da conversa.
- Adicionar avaliação automática de qualidade das respostas.